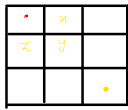


Start

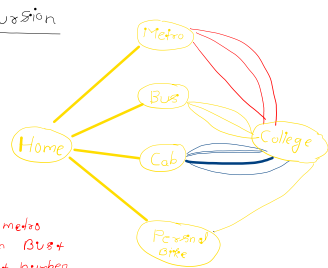


So total way to reach College from Home

= number of way from metro + number of way from Bus + number of way from Cab + number of way from Personal Bike

$$= 3+3+5+1 = 12$$

So in general total way = $x+y+z$



So total path

H H V V

H V H V

H V V H

H V D V

V V H H

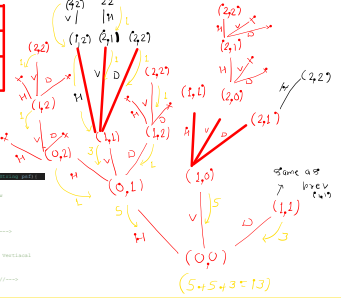
V D H H

V H H V

V H V H

D H V H

D V H H



```
System.out.println("Enter the destination row:");
int destRow = sc.nextInt(); // reached the destination row

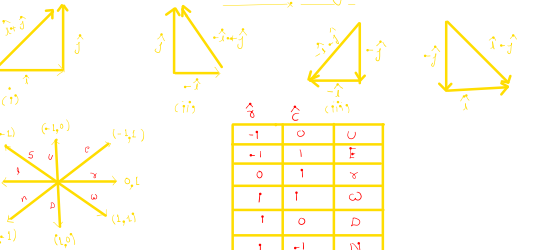
// Horizontal path
int count = 0;
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Horizontal path
        count++;
    }
}

// Vertical path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Vertical path
        count++;
    }
}

// Diagonal path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Diagonal path
        count++;
    }
}

return count;
```

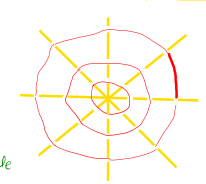
Vector theory



Maze path Code 2 Using dir array

	U	D	L	R
U	-1	0	0	0
D	0	1	0	0
L	0	0	-1	0
R	0	0	0	1

Maze path multi jump



→ for multi jump it is form a concentric circle
→ so from vector theory we only increase the radius of the circle for multiple jumps

→ $|x| = |i \cdot j|$ → only one is variable at a time because if both are variable at same time it will run in a spiral @ manner.

```
System.out.println("Enter the destination row:");
int destRow = sc.nextInt(); // reached the destination row

// Horizontal path
int count = 0;
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Horizontal path
        count++;
    }
}

// Vertical path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Vertical path
        count++;
    }
}

// Diagonal path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Diagonal path
        count++;
    }
}

return count;
```

Flood fill

→ In this problems we used the 8 direction array.
→ for avoiding the loop we will use the visited array
→ so when visit that place mark it and while backtracking unmark it
→ the algo used in it is dfs.

```
System.out.println("Enter the destination row:");
int destRow = sc.nextInt(); // reached the destination row

// Horizontal path
int count = 0;
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Horizontal path
        count++;
    }
}

// Vertical path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Vertical path
        count++;
    }
}

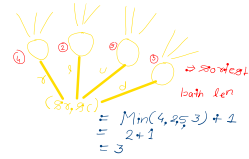
// Diagonal path
for (int i = 0; i < destRow; i++) {
    for (int j = 0; j < destCol; j++) {
        // Diagonal path
        count++;
    }
}

return count;
```

Note: same for flood fill multi jump we use visited array

just there are obstacle

Shortest path



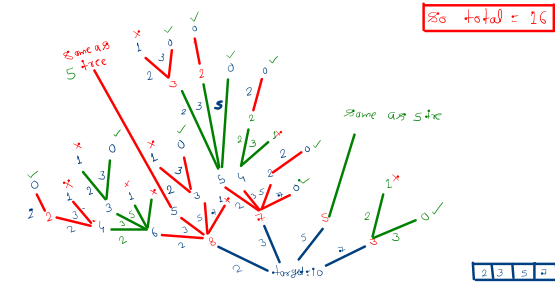
2	3	5	7
---	---	---	---

target = 10

2 2 2 2 2
2 2 3 3
2 3 5
3 4
5 5

$$\begin{array}{r} 4! / 2! \times 2! \\ 3! \\ 2! \\ 2! / 2! \end{array}$$
$$\begin{array}{r} 4! / 2! \times 2! \\ 3! \\ 2! \\ 2! / 2! \end{array}$$

16 ang



So total = 16

```

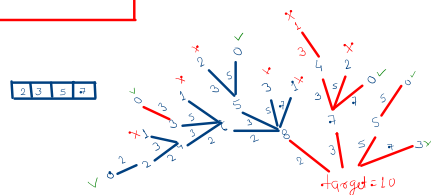
    ...
    System.out.println(pdf);
    return 1;
}

let count = 0;
for (let i = 0; i < count.length; i++) {
    if (count[i] == 0)
        count += parseInt(Math.random() * 1000000, 10) + count[i], pdf = count[i]
        + 1;
}

return count;
}

```

Coinchange Combination Inf



→ In the above problem as the coin is infinite so we can repeat the same coin while maintaining the order.

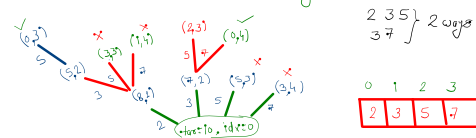
→ in the code we used a idx variable to track and maintain the order.

Note: always choose combination over permutation

Coin change Combination with Single Coin

→ as we can only use the coins only one time so we pass the $idx+1$ in above code while calling it.

→ $(0,3)$ ✓ $(1,2)$ ✗ $(2,3)$ ✗ $(0,4)$ ✓ $\left. \begin{matrix} 2 & 3 & 5 \\ 3 & 7 \end{matrix} \right\} 2 \text{ ways}$



2 3 5 } 2 ways
3 7

0	1	2	3
2	3	5	7

Leetcode 3

⇒ Code

322. Coin Change

Solved

Medium Topics Compare

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the **fewest number of coins** that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

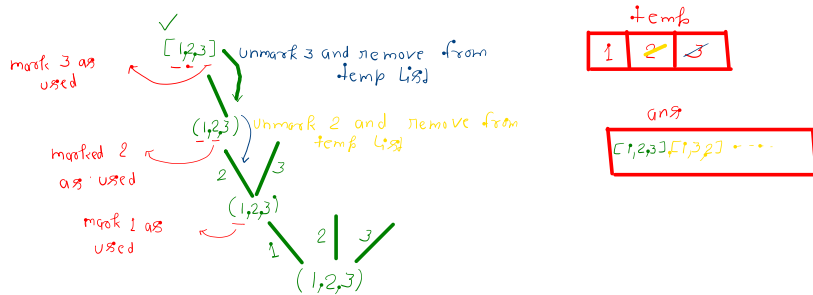
Input: `coins = [1,2,5], amount = 11`

Output:

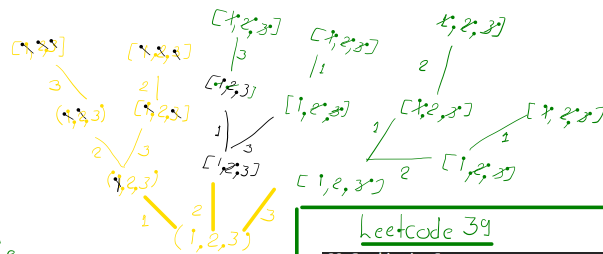
Explanation: `11 = 5 + 5 + 1`

Leetcode 46

- our faith is that we pick the element from array and they will give you its all permutation
- when we picked the array element mark it so that we can not use it again while moving up to the stack. also add in arraylist.
- as we hit the basecase copy the whole content of arraylist in our ans arraylist.
- as we going to the down of stack we unmark array element and remove from temp arraylist



So the tree looks like the below tree



Code

```
public int permutation(int []nums,int
    do, list<Integer> ans, list<Integer> ansList) {
        if (nums.length == 0) {
            list<Integer> ans = new ArrayList<Integer>();
            ans.add(nums);
            return 1;
        }
        int ans = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 11) {
                continue;
            }
            int val = nums[i];
            ansList.add(val); //add the value in the stack
            nums[i] = 11; //make the value
            ans.add(val);
            ansList.add(val); //remove the last element from
            temp list
        }
        return ans;
    }
}
```

```
public static int coinChangeCombination(int []coins, int id, int tar, list<Integer> ans, list<Integer> ansList) {
    if (tar == 0) {
        return 1;
    }
    if (tar < 0) {
        return 0;
    }
    int count = 0;
    for (int i = id; i < coins.length; i++) {
        ansList.add(coins[i]);
        count += coinChangeCombination(coins, i+1, tar-coins[i], ans, ansList);
        ansList.remove(ansList.size()-1);
    }
    return count;
}
```

Leetcode 39

39. Combination Sum

Solved

Medium Topics Compare

Given an array of **distinct** integers `candidates` and a target integer `target`, return a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`. You may return the combinations in **any order**.

The **same number** may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the **frequency** of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: `candidates = [2,3,6,7], target = 7`

Output: `[[2,2,3],[7]]`

Explanation:

2 and 3 are candidates, and `2 + 2 + 3 = 7`. Note that 2 can be used multiple times.

7 is a candidate, and `7 = 7`.

These are the only two combinations.